4,222 Introduction to Programming

Week 7: Classes

AUTHOR
Franziska Bender

PUBLISHED
April 17, 2026

In this lecture, we introduce Object-Oriented Programming (OOP) — a different way of organizing code that you have already been using without knowing it. Every time you called `df.describe()` or `df.shape`, you were using OOP. Now you will learn how to build your own objects from scratch.

By the end of this session you will know how to define a class, give it attributes and methods, and use it to bundle data and logic together into a single reusable tool.

Object Oriented Programming

Up until now, we have been writing code in a mostly "procedural" or "functional" way: writing scripts that execute top-to-bottom, and creating standalone functions to perform specific tasks. Now we look at a different way to structure code: **Object-Oriented Programming (OOP)**.

Python is inherently an object-oriented language. In OOP, we group data and the functions that operate on that data together into a single, combined unit called an Object.

To understand OOP, you need to understand two core concepts:

- **Class:** A class is a blueprint or a template. It defines what an object should look like and what it can do, but it doesn't contain any actual data itself.
- **Object:** An object is the actual "thing" created based on the class blueprint. When you create an object from a class, we call it an **instance** of that class.

In Python, almost everything you interact with is actually an object behind the scenes.

```
string_example = 'Hello'
print(type(string_example))
```

```
<class 'str'>
```

When you create `string_example`, you aren't just creating text; you are creating a specific object. It is an instance of Python's built-in `str` class.

The Anatomy of an Object: Attributes and Methods

The class blueprint gives every object two things: Attributes and Methods. The easiest way to understand them is to look at a tool you already know well: the `pandas.DataFrame`.

- **The Class**: DataFrame is the blueprint written by the pandas developers. It defines the rules for tabular data.
- **The Object**: When we read data into Python (`df = pd.read_csv(...)`), we are creating `df`, a specific instance of the DataFrame class.

Once we have our df object, we can interact with its attributes and methods:

- **Attributes**: These represent the object's data or its current state. They are properties the object has.
  - *Example:* `df.shape`, `df.columns`
  - *Note:* Attributes don't need parentheses, they are not performing an action; they are just returning a fact about the object.
- **Methods**: These are functions that belong specifically to the object. They are actions the object *can do* using its own internal data.
  - *Example:* `df.describe()`, `df.info()`
  - *Note:* Methods always use parentheses `()` because they are actively executing a block of code.

Classes

How to Create a Class

The `class` keyword

To define a class, you use the class keyword followed by the name. When naming a class in Python, use PascalCase: start each word with a capital letter, remove spaces, and don't use underscores (example: `DataFrame`) Inside the class you define how the object is built and what it can do.

```
class CentralBank:
    # Everything indented inside here belongs to the class
    pass
```

class definitions cannot be empty, but if you for some reason have a class definition with no content, put in the `pass` statement to avoid getting an error.

Now we can use the class blueprint named `CentralBank` to create objects:

```
snb = CentralBank()
```

The `__init__()` Method and `self`

So far our class is empty, let's first define some attributes of our `CentralBank`. Attributes are variables that belong to the object. They hold the object's data (like numbers, strings, or even DataFrames). To give our object attributes when it is created, we use a special method called `__init__()`.

- In Python, `__init__` stands for "initialize" and acts as the constructor.
- This method is automatically executed the moment you create a new instance of the class.
- The first argument of `__init__` must always be `self`.

Let's define two attributes of the central bank: `policy` describes how the central bank conducts monetary policy (active or passive), and `interest_rate` is the interest rate it sets.

```
class CentralBank:
    def __init__(self):                                    1
        self.policy = 'active'                             2
        self.interest_rate = 0.02
```

What is `self`?

When you define a class, you are writing code for objects that don't exist yet. You need a way to say: "When this

object is eventually created, take this data and save it inside the object itself." self is just a placeholder that refers to the specific instance being created.

Accessing Attributes

You can access these attributes outside the class using dot notation. Notice how `self` inside the class effectively becomes `snb` outside the class!

```
# create an instance of CentralBank
snb = CentralBank()

# Access the attributes
print(snb.policy)
print(snb.interest_rate)
```

```
active
0.02
```

Creating an object with specific attributes

So far, every central bank we create from the class will have exactly the same starting values — every instance gets `policy='active'` and `interest_rate=0.02`, with no way to change them at creation time. In practice, we want to be able to create different central banks with different rates and policies. We do this by adding arguments to `__init__()` (always after `self`):

```
class CentralBank:
    def __init__(self, policy, interest_rate):                    1
        self.policy = policy                                      2
        self.interest_rate = interest_rate
```

When you create an instance you must specify the values for these attributes:

```
# create an instance snb, with policy='active' and interest_rate = 0.1
snb = CentralBank(policy='active', interest_rate=0.1)
```

```
# Test: show value of the interest_rate attribute
print(snb.interest_rate)
```

```
0.1
```

Since `policy` and `interest_rate` are now required arguments, Python will raise a `TypeError` if you try to create an instance without providing them:

```
# Try creating an instance without providing the required arguments
snb = CentralBank()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[8], line 2
      1 # Try creating an instance without providing the required arguments
----> 2 snb = CentralBank()

TypeError: CentralBank.__init__() missing 2 required positional arguments: 'policy' and 'interes
```

Using Default Values

Often, you want the best of both worlds: the ability to specify exact attribute values when creating an object, but also the convenience of falling back on sensible default values if the user doesn't provide any.

Because `__init__()` is ultimately just a Python function, you can set default values for its arguments exactly the same way you do for standard standalone functions: by assigning them right in the parameter list.

```
class CentralBank:

    def __init__(self, policy='passive', interest_rate=0.02):
        self.policy = policy
        self.interest_rate = interest_rate
```

```
# If we provide no arguments, it uses the default values:
fed = CentralBank()
print(fed.policy)
print(fed.interest_rate)
```

```
passive
0.02
```

```
# If we provide arguments, it overrides the defaults:
snb = CentralBank(policy='active', interest_rate=0.05)
print(snb.policy)
print(snb.interest_rate)
```

```
active
0.05
```

What you should NOT do:

A very common mistake when learning classes is to require arguments, but then accidentally hard-code the default values inside the body of the `__init__()` method.

```
# This is an example of how NOT to provide default values
class CentralBankExample:
    def __init__(self, policy, interest_rate):
        # Python takes the inputs you provided...
        # ...but then you immediately overwrite them with hard-coded values:
        self.policy = 'passive'
        self.interest_rate = 0.02
```

```
# Create snb with a policy='active' and interest_rate=0.05...
snb = CentralBankExample(policy='active', interest_rate=0.05)

# ...the class ignores the input and forces it back to the hard-coded values:
print(snb.policy)
print(snb.interest_rate)
```

```
passive
0.02
```

Modifying Attribute Values

Once an object is created, its attributes are not permanently locked in place. As your program runs or your data updates, you will often need to change the state of your objects. There are two ways to change an attribute's value: direct access and using methods.

1. Direct Access (The Quick Way)

You can overwrite the attribute of an object directly, exactly like you would reassign a normal variable, using dot notation and the `=` sign.

```
snb = CentralBank()
print(f"Initial interest rate: {snb.interest_rate}")

# Let's directly change the value of the interest_rate attribute
snb.interest_rate = 0.05
print(f"New interest rate: {snb.interest_rate}")
```

```
Initial interest rate: 0.02
New interest rate: 0.05
```

This is like walking into the central bank and bypassing the board of directors. It is fast, and Python allows it, but it lacks any safety checks — nothing stops a user from accidentally typing `snb.interest_rate = 500` or `snb.interest_rate = "five percent"`, which would break your code later on.

2. Using Methods (The Official Way)

The preferred approach in OOP is to change attributes using methods. Instead of forcing a change directly from the outside, we ask the object to change it for us. This allows the object to act as a gatekeeper and validate the change before applying it — for example, checking that the new interest rate is actually a number, or that it doesn't exceed realistic bounds.

We haven't written any methods yet - that is the next section. But keep this motivation in mind: methods are not just for performing calculations, they are also the right way to update an object's state safely.

Methods

How to create a Method

Methods are functions that belong to a class. They define the behavior of objects created from the class.

The Basic Syntax

Methods are created *almost* exactly like functions. You use the `def` keyword, give it a name and specify arguments.

However there are two critical rules:

1. Indentation: It must be indented inside the class block. (because it belongs to the class)
2. The `self` parameter: The first argument must always be self.

Example: Let's create a method called `set_rate()` that allows us to change the value of the interest_rate attribute safely:

```
class CentralBank:
    def __init__(self, policy='passive', interest_rate=0.02):
        self.policy = policy
        self.interest_rate = interest_rate

    # Create a method set_rate():
    def set_rate(self, new_rate):                                    (1)

        self.interest_rate = new_rate                                (2)
        print(f"Rate updated to {self.interest_rate}")
```

To use a method, you use the dot notation on the specific object you created.

```
snb = CentralBank()
# Call the method:
snb.set_rate(new_rate=0.03)
print(snb.interest_rate)
```

```
Rate updated to 0.03
0.03
```

This is why you need `self` as the first argument. A method is not a function, you don't apply a function like `set_rate(snb, 0.03)`, your object `snb` knows how to use this function on itself `snb.set_rate(0.03)`.

Note that there is no `return` in this method. It doesn't return a value, it sets the value of an attribute. Methods can also return things just like functions, we'll see examples later

Why Methods are Better than Direct Access

Setting new values through a method (rather than just typing `snb.interest_rate = 0.03`) is all about safety and automation. This is especially vital when collaborating with others or cleaning complex datasets:

- Gatekeeping: A method allows you to catch user errors before they break your code.
- Feedback: You can immediately provide print messages or warnings to guide the user on what they should do instead.
- Fail-safes: It ensures that the object always remains in a valid, logical state.

Example: Adding Safety Checks

Let's upgrade our `set_rate` method. We want to ensure two things:

1. The new interest rate must actually be a number.
2. If the rate is negative (which is unconventional but possible), we want to accept it but print a warning.

```
class CentralBank:

    def __init__(self, policy='passive', interest_rate=0.02):
        self.policy = policy
        self.interest_rate = interest_rate
```

```
    def set_rate(self, new_rate):
        # 1. Check if the input is a number (int or float)
        if not isinstance(new_rate, (int, float)):
            print(f"Error: {new_rate} is not a number. Rate remains {self.interest_rate}.")
            return  # Exit the method early

        # 2. Check for unconventional negative rates
        if new_rate < 0:
            print("Warning: Negative rates are unconventional!")

        # 3. If it passed the 'number' check, update the value
        self.interest_rate = new_rate
        print(f"Policy Update: Rate is now {self.interest_rate}")
```

Now let's see our safety checks in action:

```
snb = CentralBank()

# Trying to pass a string instead of a number
snb.set_rate('5 percent')
```

```
Error: 5 percent is not a number. Rate remains 0.02.
```

```
# Passing a valid, but negative number
snb.set_rate(-0.01)
```

```
Warning: Negative rates are unconventional!
Policy Update: Rate is now -0.01
```

Roles of Methods

We've seen an example of a method that sets the value of an attribute (`set_rate`). That is one specific use case, but methods generally define everything an object can do. Two other particularly important roles for methods are:

1. **Computation & Analysis:** Instead of just storing data, the object can process its own internal data (and combining it with external inputs) to give you an answer.
2. **Reporting and Visualization:** Methods can be used to display the object's data in a way that is easy to read or visualize. This is where your `matplotlib` skills will come in.

Example 1: (Computation & Analysis)
Let's add a method so the CentralBank can calculate the real interest rate when given an inflation rate.

```
class CentralBank:

    def __init__(self, policy='passive', interest_rate=0.02):
        self.policy = policy
        self.interest_rate = interest_rate

    # Method to set an attribute
    def set_rate(self, new_rate):
        if not isinstance(new_rate, (int, float)):
            print(f"Error: {new_rate} is not a number. Rate remains {self.interest_rate}.")
            return

        if new_rate < 0:
            print("Warning: Negative rates are unconventional!")

        self.interest_rate = new_rate
        print(f"Policy Update: Rate is now {self.interest_rate}")


    # NEW: Method for computation
    def calculate_real_rate(self, inflation):
        # This method performs a calculation using internal data and an external input
        return self.interest_rate - inflation
```

```
snb = CentralBank()
print(f"The nominal interest rate is {snb.interest_rate}")
```

```
# We pass the external input (inflation) into our new method:
real_interest_rate = snb.calculate_real_rate(inflation=0.01)

print(f"The real interest rate is {real_interest_rate}")
```

```
The nominal interest rate is 0.02
The real interest rate is 0.01
```

Note on `return`: Notice that we use `return` in this method. This is because we do not want to permanently change the central bank's actual interest rate. We just want the object to perform a calculation and hand the result back to us to use as a variable.

Example 2: (Reporting and Visualization)
We want to print an overview of the central bank's policy, the type and the interest rate

```
class CentralBank:

    def __init__(self, policy='passive', interest_rate=0.02):
        self.policy = policy
        self.interest_rate = interest_rate

    # Method to set an attribute
    def set_rate(self, new_rate):
        if not isinstance(new_rate, (int, float)):
            print(f"Error: {new_rate} is not a number. Rate remains {self.interest_rate}.")
            return

        if new_rate < 0:
            print("Warning: Negative rates are unconventional!")

        self.interest_rate = new_rate
        print(f"Policy Update: Rate is now {self.interest_rate}")


    # Method for computation
    def calculate_real_rate(self, inflation):
```

```
        # This method performs a calculation using internal data and an external input
        return self.interest_rate - inflation


    # NEW: Method for reporting / visualization
    def display_status(self):
        # A method that prints a pretty summary of the bank's current state
        print("-" * 20)
        print(f"CENTRAL BANK REPORT")
        print(f"Policy Type: {self.policy.upper()}")
        print(f"Current Rate: {self.interest_rate:.2%}")
        print("-" * 20)
```

Now, whenever we want to see what is going on with our object, we just call this one simple method:

```
snb = CentralBank()
snb.display_status()
```

```
--------------------
CENTRAL BANK REPORT
Policy Type: PASSIVE
Current Rate: 2.00%
--------------------
```

Exercise: The Central Bank Policy Rule

To consolidate these concepts, we will build a functional `CentralBank` class that manages its own state and responds dynamically to economic shocks.

This exercise demonstrates how attributes define an object's internal rules (its "personality"), and how methods allow that object to process external data to make policy decisions and update its state.

The Economic Rule:

- If inflation is above the target, the bank must increase the interest rate.
- If inflation is below the target, the bank must decrease the interest rate.

- An ‘active’ central bank adjusts rates aggressively: by 0.01 (100 basis points).
- A ‘passive’ central bank adjusts rates cautiously: by 0.0025 (25 basis points).

How to implement this step by step:

1. Define the class and constructor:
   - Add another attribute ‘target’ which is the banks inflation target (set a default of 0.02).
2. Define a new method called `react_to_inflation`
   - Use an `if` / `else` statement to check `self.policy` and determine the adjustment size (0.01 or 0.0025).
   - Use another `if` / `elif` statement to compare `current_inflation` to the target, and then add or subtract the adjustment from `self.interest_rate`.
   - Add a `print()` statement at the end summarizing the action taken and the new rate.
3. Test your class
   - Create a “Hawkish Fed” that is active
   - Create a “Dovish ECB” that is passive
   - Pass a current_inflation value of 0.05 to both banks using their react_to_inflation method and observe how their internal states diverge.

```
class CentralBank:
    def __init__(self, policy='passive', interest_rate=0.02, target=0.02):          1
        self.policy = policy
        self.interest_rate = interest_rate
        self.target = target

    def react_to_inflation(self, current_inflation):                                 2
        # 1. Determine the 'strength' of the reaction based on the policy attribute
        if self.policy == 'active':                                                  3
            adjustment = 0.01   # 100 basis points
        else:
            adjustment = 0.0025 # 25 basis points

        # 2. Apply the adjustment logic
        if current_inflation > self.target:                                          4
            self.interest_rate += adjustment
```

```
            action = f"Hiked by {adjustment:.2%}"
        elif current_inflation < self.target:
            self.interest_rate -= adjustment
            action = f"Cut by {adjustment:.2%}"
        else:
            action = "Maintained"

        print(f"[{self.policy.upper()} BANK] Inflation: {current_inflation:.1%}, "
              f"Target: {self.target:.1%} -> {action}. New Rate: {self.interest_rate:.2%}")
```

```
# The Inflation Data
current_inflation = 0.05

# Creating two different objects from the same blueprint (class)
fed = CentralBank(policy='active', interest_rate=0.04, target=0.02)
ecb = CentralBank(policy='passive', interest_rate=0.04, target=0.02)

# Comparing their reactions to the exact same shock
fed.react_to_inflation(current_inflation)
ecb.react_to_inflation(current_inflation)
```

```
[ACTIVE BANK] Inflation: 5.0%, Target: 2.0% -> Hiked by 1.00%. New Rate: 5.00%
[PASSIVE BANK] Inflation: 5.0%, Target: 2.0% -> Hiked by 0.25%. New Rate: 4.25%
```

Create a Module

As your projects grow in complexity, keeping your class definitions and your data analysis in the same file becomes difficult to manage. Professional Python developers organize their code into modules.

A module is simply a `.py` file containing your classes and functions. You can then “borrow” those tools in your main analysis script or Jupyter/Quarto notebook.

- Create a new file `monetary_policy.py` (make sure the extension is `.py`)
- Copy the final, working version of our `CentralBank` class and paste it into that new file.
- Save the file. You have just created your first Python module!

How to import your own Classes

Now that the tool exists in its own file, we can import it into our current notebook or any other notebook or script. There are three common ways to bring it into your workspace:

1. Import the Entire Module

The most basic approach is to import the whole file. To create an object, you must use dot notation to tell Python: “Look inside the monetary_policy module, and find the CentralBank class.”

```
import monetary_policy

# Create an instance: You must prefix the class name with the module name
fed = monetary_policy.CentralBank(policy='active', interest_rate=0.01)
print(fed.interest_rate)
```

```
0.01
```

2. Import the Entire Module with an Alias

Typing `monetary_policy` over and over gets tedious. It is standard practice to give the module a shorter nickname (an alias) when you import it.

This is the exact same logic we use when we type `import pandas as pd`!

```
import monetary_policy as mp

# Now we just use the 'mp.' prefix
fed = mp.CentralBank(policy='active', interest_rate=0.01)
print(fed.interest_rate)
```

```
0.01
```

3. Import a Specific Class Directly

If you know you only need one specific class from a large module, you can pull it directly into your workspace using the `from ... import ...` syntax.

Because you imported the class specifically, you no longer need the module prefix at all. You can use it just like a built-in Python function.

```
# In your main analysis file
from monetary_policy import CentralBank

# Now you can use it just like a built-in tool
fed = CentralBank(policy='active', interest_rate=0.01)
fed.react_to_inflation(current_inflation = 0.2)
```

```
[ACTIVE BANK] Inflation: 20.0%, Target: 2.0% -> Hiked by 1.00%. New Rate: 2.00%
```

Best Practices: Structuring Your Module

If you are creating a `.py` file to hold the classes and functions you use in your project, there are a few structural conventions that make a massive difference for organization and readability.

1. **Imports:** Put any libraries the class needs (like `import pandas as pd`) at the very top.
2. **The Class:** Define your class
3. **Docstrings:** Use docstrings (triple quotes) `""" """` to explain what the class and the methods do. This is helpful because when you hover over the class the text will pop up as a “help” hint.

```
class CentralBank():
    """
    A class to represent a central bank and simulate its monetary policy decisions.

    Attributes
    ----------
    policy : str
        The bank's policy stance, either 'active' or 'passive'.
    interest_rate : float
        The current nominal interest rate set by the bank.
    target : float
        The central bank's target inflation rate.
    """
```

```
    def __init__(self, policy='passive', interest_rate=0.02, target=0.02):
        """
        Initializes the CentralBank with a policy stance, initial rate, and target.
        """
        # The rest of the code

    def react_to_inflation(self, current_inflation):
        """
        Adjusts the interest rate based on the current inflation rate.

        An 'active' bank adjusts rates by 100 basis points (0.01), while a
        'passive' bank adjusts by 25 basis points (0.0025).

        Parameters
        ----------
        current_inflation : float
            The current economic inflation rate to react to.
        """
        # The rest of the code
```

Add some docstrings to your class in `monetary_policy.py`, save it and then import the Class again:

```
from monetary_policy import CentralBank
snb = CentralBank()
```

You can use the `help()` function on your class or on a method and it will grab those internal strings and print a manual for them.

```
help(CentralBank)
```

```
Help on class CentralBank in module monetary_policy:

class CentralBank(builtins.object)
 |  CentralBank(policy='passive', interest_rate=0.02, target=0.02)
```

```
 |
 |  A class to represent a central bank and simulate its monetary policy decisions.
 |
 |  Attributes
 |  ----------
 |  policy : str
 |      The bank's policy stance, either 'active' or 'passive'.
 |  interest_rate : float
 |      The current nominal interest rate set by the bank.
 |  target : float
 |      The central bank's target inflation rate.
 |
 |  Methods defined here:
 |
 |  __init__(self, policy='passive', interest_rate=0.02, target=0.02)
 |      Initializes the CentralBank with a policy stance, initial rate, and target.
 |
 |  react_to_inflation(self, current_inflation)
 |      Adjusts the interest rate based on the current inflation rate.
 |
 |      An 'active' bank adjusts rates by 100 basis points (0.01), while a
 |      'passive' bank adjusts by 25 basis points (0.0025).
 |
 |      Parameters
 |      ----------
 |      current_inflation : float
 |          The current economic inflation rate to react to.
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors defined here:
 |
 |  __dict__
 |      dictionary for instance variables
 |
 |  __weakref__
 |      list of weak references to the object
```

Sometimes that's a bit too much information, if you want to see the docstring for the class only, you can access it using `MyClass.__doc__`. This is a special hidden attribute that is created automatically

```
print(CentralBank.__doc__)
```

```
A class to represent a central bank and simulate its monetary policy decisions.

Attributes
----------
policy : str
    The bank's policy stance, either 'active' or 'passive'.
interest_rate : float
    The current nominal interest rate set by the bank.
target : float
    The central bank's target inflation rate.
```

Similarly you can use `help()` for a specific method of the class

```
help(CentralBank.react_to_inflation)
```

```
Help on function react_to_inflation in module monetary_policy:

react_to_inflation(self, current_inflation)
    Adjusts the interest rate based on the current inflation rate.

    An 'active' bank adjusts rates by 100 basis points (0.01), while a
    'passive' bank adjusts by 25 basis points (0.0025).

    Parameters
    ----------
    current_inflation : float
        The current economic inflation rate to react to.
```

or access the docstring of a method using `MyClass.my_method.__doc__`

```
print(CentralBank.react_to_inflation.__doc__)
```

```
Adjusts the interest rate based on the current inflation rate.

An 'active' bank adjusts rates by 100 basis points (0.01), while a
'passive' bank adjusts by 25 basis points (0.0025).

Parameters
----------
current_inflation : float
    The current economic inflation rate to react to.
```

4. **No "Loose" Code:** Avoid having code outside of a class or function in your .py file. They will execute the second you type import, which can be very annoying.
   - Performance: If your loose code includes a heavy data calculation your analysis script will hang for 30 seconds every time you try to import your class.
   - Variable Pollution: If you define a variable `x = 10` as loose code in the module, and you have another `x = 50` in your main script, it can lead to confusing bugs where variables overwrite each other.

5. **The Gatekeeper:** `if __name__ == "__main__"` : At the bottom of your module you can have testing code

Every Python file has a built-in variable called `__name__` . Its value changes depending on how you are using the file:

1. When you run the file directly: Python sets `__name__` to `"__main__"` . The condition is true, and the code inside the block runs.
2. When you import the file: Python sets `__name__` to the filename (e.g., "monetary_policy"). The condition is false, and the code inside the block is skipped.

Implementation Template

```python
# --- monetary_policy.py ---

class CentralBank:
    def __init__(self, target=0.02):
        self.target = target

    def announce(self):
        print(f"Our inflation target is {self.target:.1%}")

# The Gatekeeper
if __name__ == "__main__":
    # This code ONLY runs if you run this file directly.
    # It will NOT run when you 'from monetary_policy import CentralBank'
    print("Running local test...")
    test_bank = CentralBank(target=0.03)
    test_bank.announce()
```

This is useful for

1. **Module Testing:** You can verify your code works
2. **Documentation by Example:** If a co-author opens your module, they can scroll to the bottom to see a working example of how to work with your Classes

Note: we call a `.py` a module when the purpose is to hold classes/functions things that are meant to be imported. Of course you can also have a `.py` file where you run your analysis, this is called a script. A script is meant to be run directly so you don't need to worry about 'loose' code or the `if __name__ == "__main__"` block.